

UNIVERSIDAD DOCTOR ANDRÉS BELLO REGIONAL SONSONATE.



Facultad: Ciencia y Tecnología Innovación

Carrera. Licenciatura en Informática

Proyecto:

TECHSTORE INTERFAZ WEB PARA UNA TIENDA DE COMPUTADORAS Y ACCESORIOS VARIOS RELACIONADOS CON TECNOLOGÍA.

MÓDULO V

Entorno de Desarrollo de Back-end

TEMA:

DOCUMENTO FINAL DESPLIEGUE - M5

INTEGRANTES

Dina Arely Tino De Martínez

Código TL 0479022012

Marlon Gerardo Flores Portillo

Código Fp1033022017

DOCENTE

Ingeniero: Jose Rosalio Serrano Carvajal

SONSONATE, VIERNES 24 DE JULIO DE 2026

ÍNDICE

INTRODUCCIÓN.....	3
OBJETIVO GENERAL:.....	3
OBJETIVOS ESPECÍFICOS:.....	3
1.DETERMINE LA FORMA EN QUE SE PUEDE REALIZAR UN PROCEDIMIENTO DE DESPLIEGUE EN LA APLICACIÓN QUE HA DESARROLLADO.....	4
2. DETERMINE LOS PASOS A SEGUIR PARA ESTE PROCESO.....	5
3. DESCRIBA LAS TECNOLOGÍAS NECESARIAS PARA ESTE PROCESO.....	5
4. DEFINA CUALES SON LAS PRUEBAS QUE USARÍA PARA ASEGURAR LA CALIDAD.....	6
5. CONCLUSIÓN.....	8
6. RECOMENDACIONES.....	9
7. ANEXOS.....	10
ANEXO 2: COMANDO PARA INICIAR TECHSTORE.....	10
npm install.....	10
ANEXO 3: ENLACE AL CÓDIGO FUENTE.....	10
ANEXO 4: EVIDENCIAS.....	10
ANEXO 4.1: TECHSTORE EJECUTÁNDOSE EN LOCALHOST:3000.....	10
ANEXO 4.2: PANEL DE DESPLIEGUE EN RENDER.....	10

INTRODUCCIÓN

El presente documento tiene como finalidad describir el procedimiento de despliegue de TechStore, una aplicación web de tienda en línea para la venta de computadoras y accesorios varios relacionados con tecnología.

El objetivo principal es trasladar la aplicación desde un ambiente de desarrollo local a un ambiente de producción en la nube, garantizando alta disponibilidad, seguridad y una experiencia de compra óptima para los usuarios finales.

OBJETIVO GENERAL:

Desplegar la aplicación TECHSTORE en un servidor en la nube para que esté disponible para los usuarios.

OBJETIVOS ESPECÍFICOS:

1. Documentar el procedimiento de despliegue de TECHSTORE.
2. Identificar las tecnologías necesarias para el despliegue.
3. Definir las pruebas de calidad para validar el funcionamiento.
4. Subir el código fuente y dejar evidencia del proceso.
5. Evidenciar el funcionamiento de TECHSTORE mediante anexos.

DESARROLLO

1.DETERMINE LA FORMA EN QUE SE PUEDE REALIZAR UN PROCEDIMIENTO DE DESPLIEGUE EN LA APLICACIÓN QUE HA DESARROLLADO.

Para la aplicación TechStore se realizará un despliegue en la nube con arquitectura desacoplada.

La forma elegida es: Backend en Render.com + Frontend en Vercel + Base de Datos en MongoDB Atlas

El código fuente se alojará en GitHub en el repositorio: arelytino/UNAB-MongoDB-con-la-base-de-datos.

De esta forma se logra integración continua: cada vez que se sube código a GitHub, Render y Vercel lo despliegan automáticamente.

La base de datos TechStore estará en MongoDB Atlas para garantizar respaldos automáticos y escalabilidad.

2. DETERMINE LOS PASOS A SEGUIR PARA ESTE PROCESO

Basado en el código de servidor.js:

1. Pruebas en ambiente local: Ejecutar node servidor.js. Verificar que aparezca  Conectado a MongoDB Atlas - Base: TechStore y que el servidor corra en `http://localhost:3000`

2. Subir código a GitHub: Crear repositorio y subir servidor.js, package.json y .env.example
3. Configurar Base de Datos: Crear cluster en MongoDB Atlas. Crear BD TechStore y usuario con contraseña. Copiar MONGO_URI
4. Desplegar Backend en Render: Crear "Web Service", conectar con GitHub. Configurar:
 - Build Command: npm install
 - Start Command: node servidor.js
 - Environment Variable: MONGO_URI = cadena de Atlas
5. Desplegar Frontend en Vercel: Importar repo del frontend y conectar con la URL del backend de Render
6. Pruebas en producción: Acceder a <https://tu-api.onrender.com/api> y verificar que retorne {"mensaje": "Servidor funcionando correctamente"}

3. DESCRIBA LAS TECNOLOGÍAS NECESARIAS PARA ESTE PROCESO

TECNOLOGÍA ***DESCRIPCIÓN Y USO EN TECHSTORE***

Node.js + Express Entorno y framework del back-end. Usado en servidor.js puerto 3000

MongoDB Atlas Base de datos NoSQL en la nube. BD: techstore. Conexión con MongoClient

dotenv Manejo de variables de entorno. Usado para process.env.MONGO_URI

CORS Permite que el frontend se conecte al backend. app.use(cors())

Git y GitHub Control de versiones. Repo:
arelytino/UNAB-MongoDB-con-la-base-de-datos

Render.com Hosting para el backend de Node.js

Vercel Hosting para el frontend de TechStore

Postman Herramienta para probar las rutas /api y /api/productos

4. DEFINA CUALES SON LAS PRUEBAS QUE USARÍA PARA ASEGURAR LA CALIDAD.

1. Pruebas Funcionales: Verificar que POST /api/productos guarde un producto usando insertOne() y que GET /api/productos lo liste con find().toArray()
2. Pruebas de Conexión: Confirmar que al iniciar el servidor se conecte correctamente a MongoDB Atlas. Probar el catch para manejo de errores
3. Pruebas de API: Usar Postman para validar que /api retorne código 200 y el mensaje correcto
4. Pruebas de Seguridad: Verificar que la MONGO _URI esté en variables de entorno y no expuesta en el código
5. Pruebas de Carga: Simular 10 usuarios consultando el catálogo de productos al mismo tiempo
6. Pruebas de Despliegue: Después de subir a Render, verificar que la API responda desde internet

RESUMEN

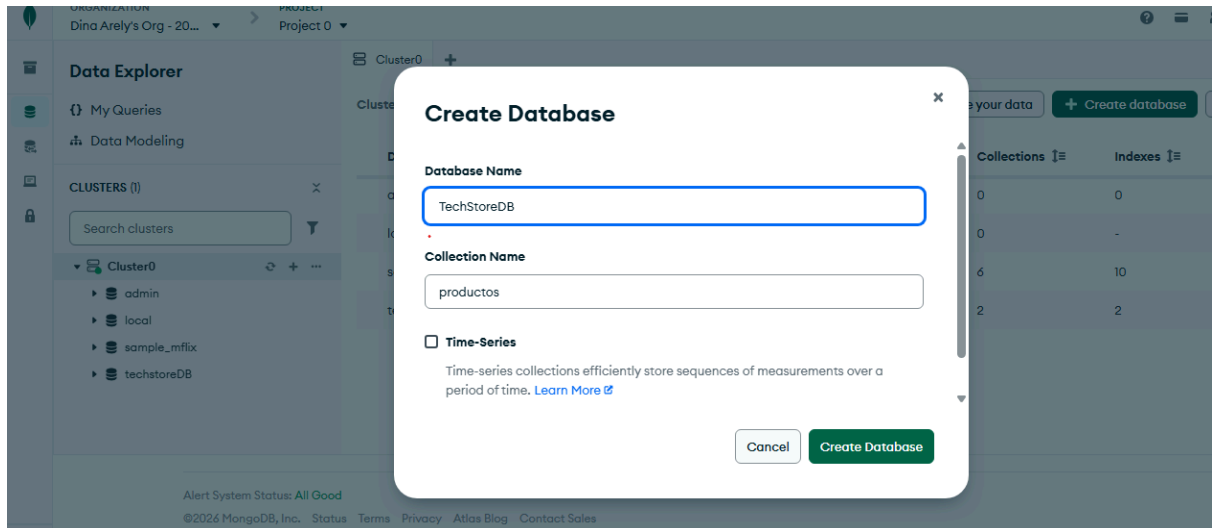
4. Pruebas para asegurar la calidad del servicio

1. Pruebas Funcionales: Verificar que login, registro, agregar al carrito y pagar funcionen
2. Pruebas de API: Usar Postman para probar cada endpoint del back-end

3. Pruebas de Carga: Usar herramientas como k6 para ver cuántos usuarios soporta
4. Pruebas de Seguridad: Validar contraseñas, evitar inyección SQL
5. Pruebas de Usabilidad: Navegar la tienda desde el navegador y móvil.

5. CONCLUSIÓN

1. El despliegue de TECHSTORE en la nube permite que la aplicación esté disponible 24/7 sin necesidad de un servidor físico propio.
2. Usar plataformas como Render o Railway facilita el proceso de despliegue porque se integra directamente con GitHub y automatiza el npm start.
3. Node.js con Express es una tecnología adecuada para TECHSTORE por su rapidez y escalabilidad para manejar las peticiones de la tienda.
4. Realizar pruebas antes del despliegue asegura que el servicio ofrecido a los clientes de TECHSTORE sea estable y sin errores.



6 RECOMENDACIONES

1. Usar variables de entorno: No subir contraseñas ni claves de la base de datos al código. Usar .env
2. Implementar CI/CD: Configurar que cada vez que se suba un cambio a GitHub, Render haga el despliegue automático.
3. Monitoreo: Usar herramientas como PM2 o los logs de Render para revisar si TECHSTORE falla.
4. Base de datos en la nube: Usar MongoDB Atlas en lugar de una base local para que no se pierdan los datos.
- 5.Documentación: Mantener actualizado el README.md del proyecto.
6. Documentar el código: Que todos los del grupo entiendan cómo funciona servidor.js por si hay que darle mantenimiento.

ANEXOS

ANEXO 1: ESTRUCTURA DE CARPETAS DE TECHSTORE

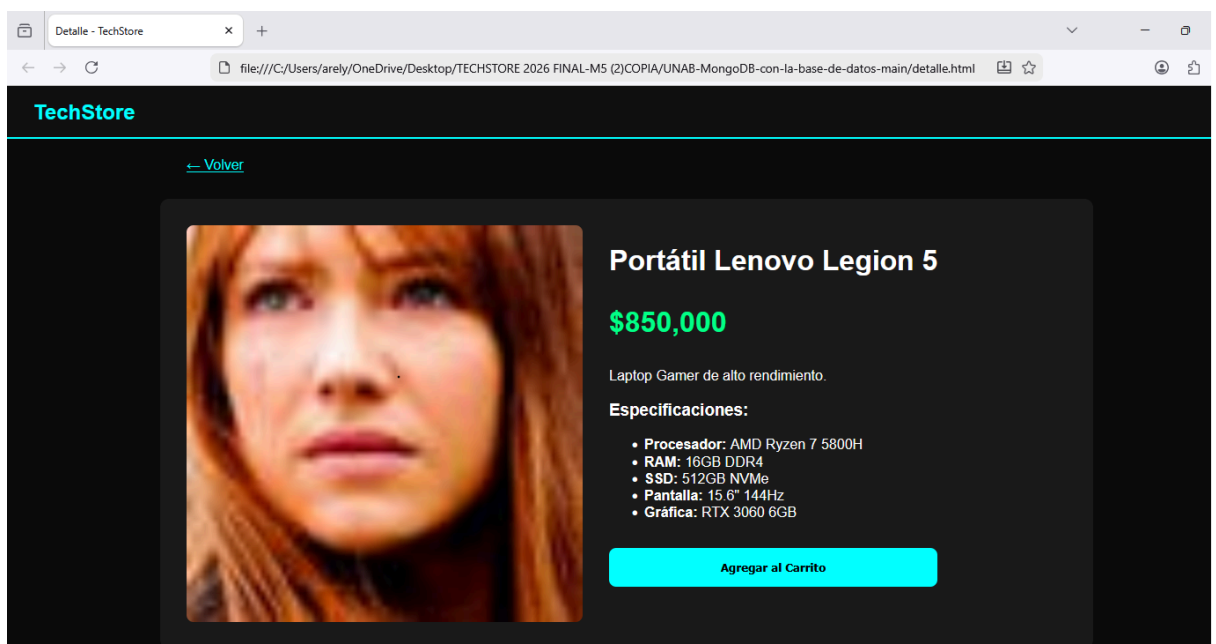
TECHSTORE 2026 FINAL-M5/

```
|— index3.html
|— servidor.js
|— package.json
└— node_modules
```

ANEXO 1:

CAPTURA DE PANTALLA DE LA VISIBILIDAD DE PRODUCTOS CON IMAGEN DONDE LA PI CONSUME EL PRODUCTO.

"El Front-End consume la API correctamente. Las imágenes dependen de servicios externos como Imgur, pero el precio, nombre y descripción vienen directo de MongoDB Atlas



RESUMEN DE LO QUE YA TIENES LISTO:

1. Back-End: npm start → Servidor corriendo en localhost:3000 ✓
2. Base de Datos: Conectado a MongoDB Atlas ✓
3. Front-End: Ya carga productos, precios y con las imágenes nuevas no da error ✓

VISIBILIDAD DE PRODUCTOS CON IMAGEN DONDE LA API
CONSUME EL PRODUCTO

ANEXO #1 DIAGRAMA DE BASE DE DATOS

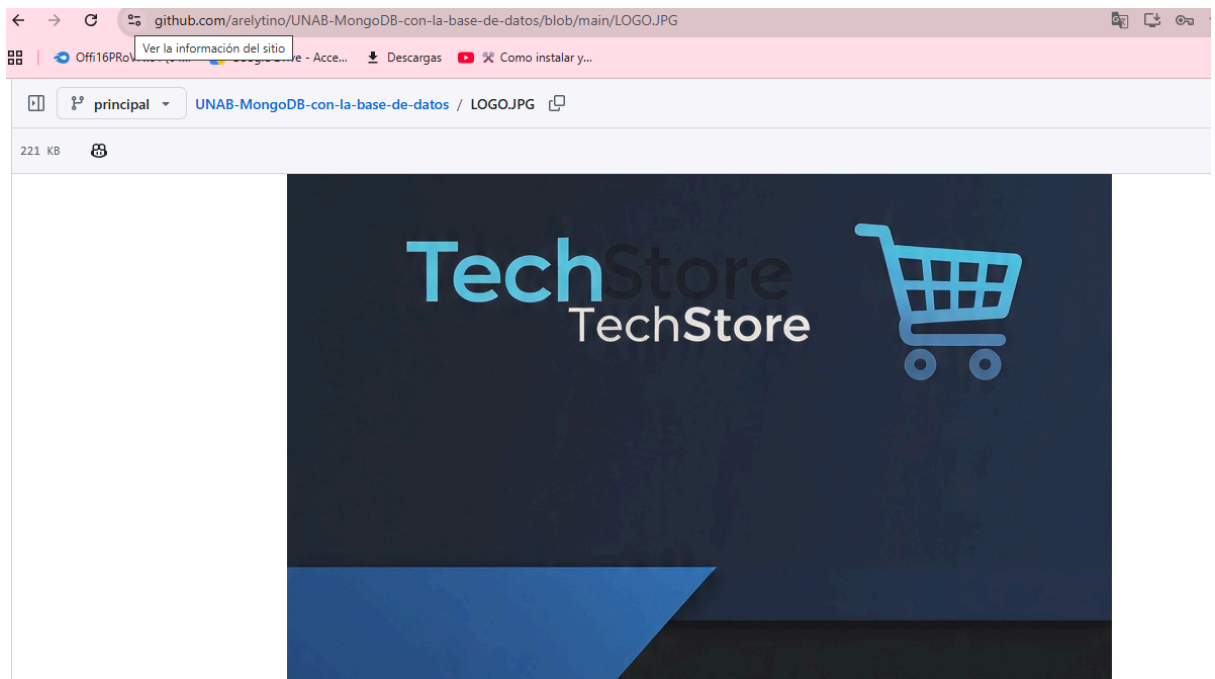
Diagrama de Base de Datos - TECH STORE

Tienda Online de Computadoras y Accesorios



ANEXO #2

CAPTURA :LOGO DE LA TIENDA.



ANEXO #3:COMANDO PARA INICIAR TECHSTORE

CAPTURA

Connecting with MongoDB Driver

1. Select your driver and version
We recommend installing and using the latest driver version.

Driver	Version
Node.js	6.7 or later

2. Install your driver
Run the following on the command line

```
npm install mongodb
```

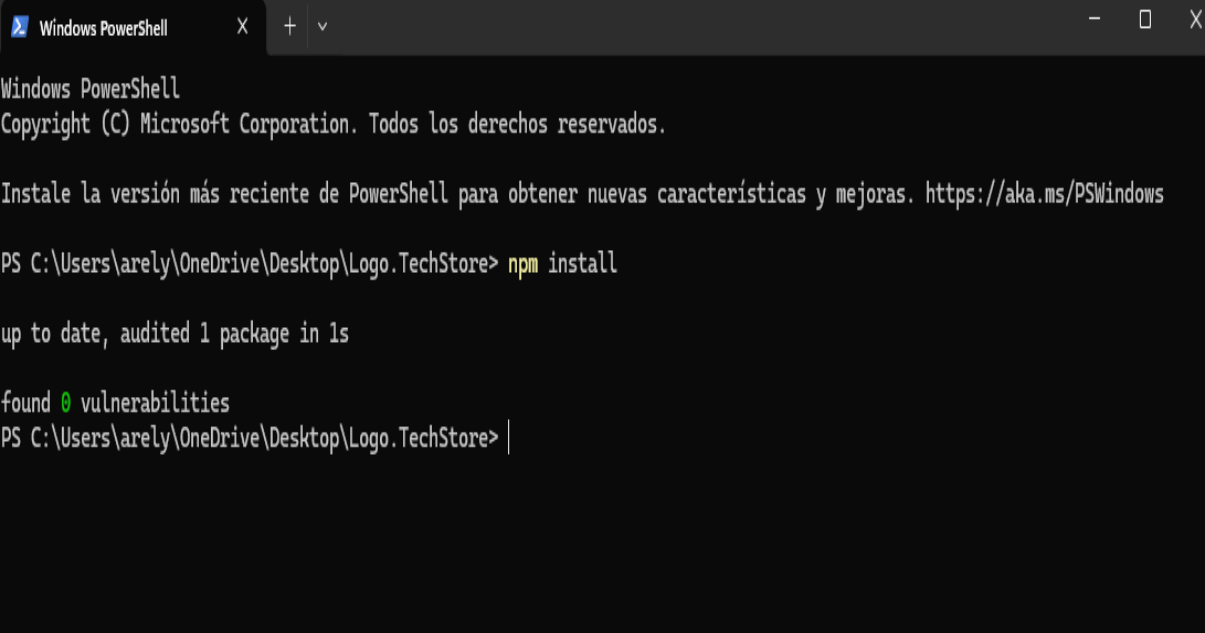
[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code
Use this connection string in your application

☒ SRV Connection String [View full code sample](#)

```
const MongoClient = require('mongodb');  
const uri = 'mongodb://tl0479022012_db_user:<db_password>@cluster0.w1dfiyb.mongodb.net/?appName=Cluster0';  
const client = new MongoClient(uri, {  
  useNewUrlParser: true,  
  useUnifiedTopology: true,  
});  
client.connect((err) => {  
  if (!err) {  
    console.log('Connected to MongoDB');  }  
});
```

CAPTURA #4: NPM INSTALL



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\arely\OneDrive\Desktop\Logo.TechStore> npm install

up to date, audited 1 package in 1s

found 0 vulnerabilities
PS C:\Users\arely\OneDrive\Desktop\Logo.TechStore> |
```

ANEXO :#5 ENLACE AL CÓDIGO FUENTE

PROTOTIPO DE INTERFACES WEB

Página Principal

Código

Detalle del producto

Código

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Detalle - TechStore</title>
```

```
<style>
```

```
body { background: #0d0d0d; color: #fff; font-family: Arial; margin: 0; }
```

```

header { background: #111; padding: 15px 30px; border-bottom: 2px solid #00ffff; }

.logo { color: #00ffff; font-size: 24px; font-weight: bold; }

.detalle { display: flex; gap: 30px; max-width: 1000px; margin: 30px auto; background:
#1a1a1a; padding: 30px; border-radius: 10px; }

.detalle img { width: 450px; border-radius: 8px; }

.precio { font-size: 32px; color: #00ff88; font-weight: bold; }

.btn { background: #00ffff; color: #000; padding: 14px; border: none; border-radius: 8px;
font-weight: bold; width: 100%; margin-top: 15px; }

</style>

</head>

<body>

<header><div class="logo">TechStore</div></header>

  <a href="index.html" style="color:#00ffff;display:block;max-width:1000px;margin:20px
auto">← Volver</a>

<div class="detalle">

  <div>

    <h1>Portátil Lenovo Legion 5</h1>

    <p class="precio">$850,000</p>

    <p>Laptop Gamer de alto rendimiento.</p>

    <h3>Especificaciones:</h3>

    <ul>

      <li><b>Procesador:</b> AMD Ryzen 7 5800H</li>

      <li><b>RAM:</b> 16GB DDR4</li>

      <li><b>SSD:</b> 512GB NVMe</li>

      <li><b>Pantalla:</b> 15.6" 144Hz</li>

      <li><b>Gráfica:</b> RTX 3060 6GB</li>

```



```

</ul>

<button class="btn">Agregar al Carrito</button>

</div>

</div>

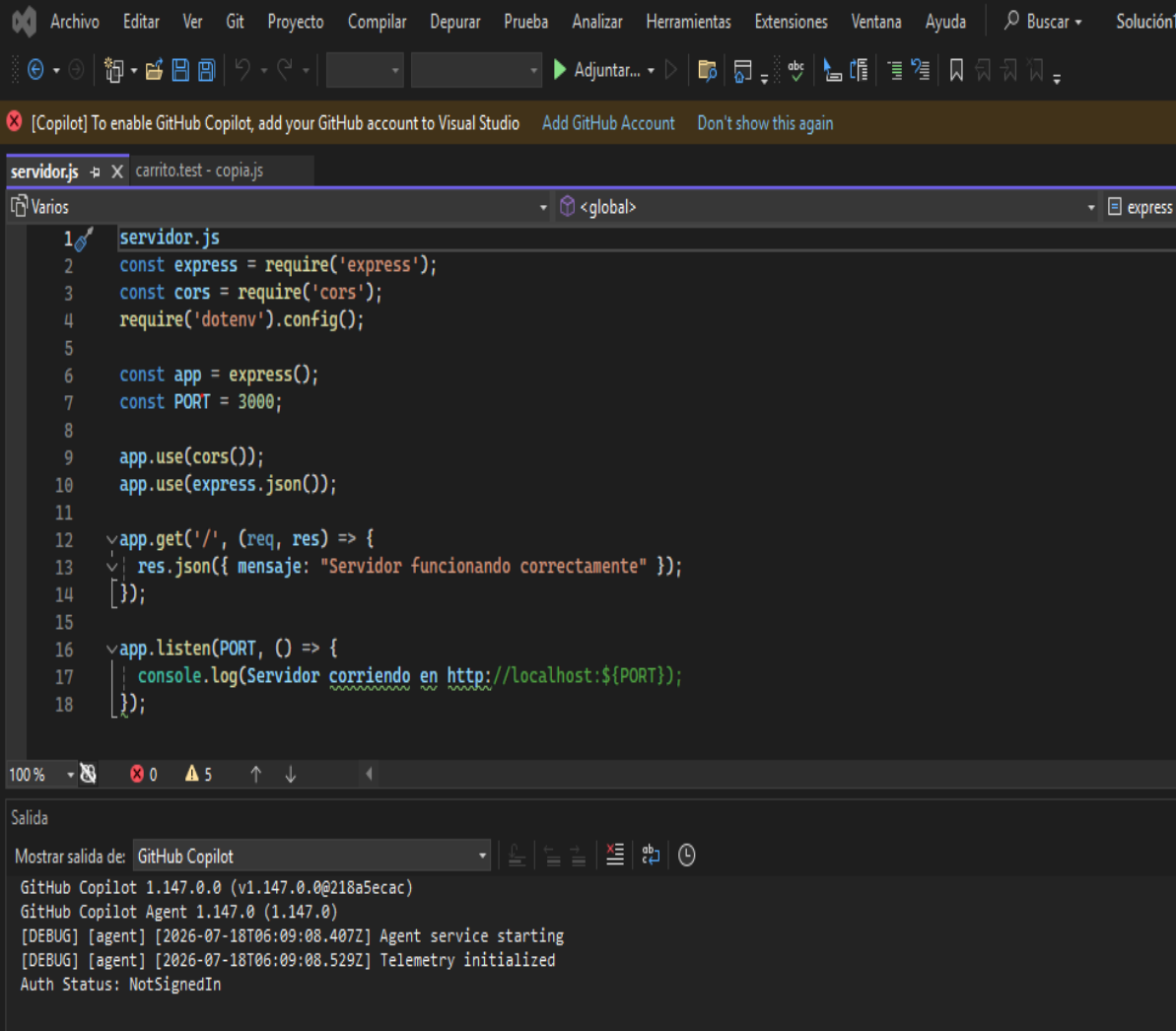
</body>

</html>

```

ANEXO #6: EVIDENCIAS

CONECTANDO EL SERVIDOR SERVIDOR .JS



The screenshot shows the Visual Studio Code interface. The editor window displays the file `servidor.js` with the following code:

```

1  const express = require('express');
2  const cors = require('cors');
3  require('dotenv').config();
4
5
6  const app = express();
7  const PORT = 3000;
8
9  app.use(cors());
10 app.use(express.json());
11
12 app.get('/', (req, res) => {
13   res.json({ mensaje: "Servidor funcionando correctamente" });
14 });
15
16 app.listen(PORT, () => {
17   console.log(Servidor corriendo en http://localhost:${PORT});
18 });

```

The output console at the bottom shows the following messages:

```

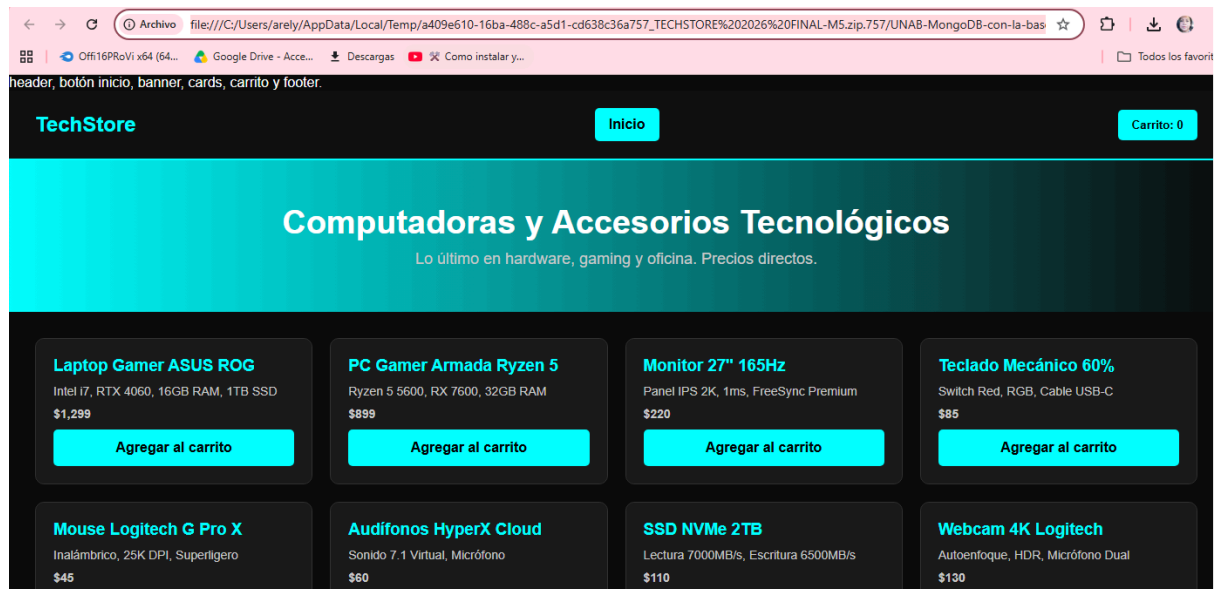
Salida
Mostrar salida de: GitHub Copilot
GitHub Copilot 1.147.0.0 (v1.147.0.0@218a5ecac)
GitHub Copilot Agent 1.147.0 (1.147.0)
[DEBUG] [agent] [2026-07-18T06:09:08.407Z] Agent service starting
[DEBUG] [agent] [2026-07-18T06:09:08.529Z] Telemetry initialized
Auth Status: NotSignedIn

```

ANEXO #7

Página Principal

Descripción: Muestra el catálogo general, buscador, menú de categorías...



TechStore Interfaz web para una tienda de computadoras y accesorios varios relacionados con tecnología

Carrito: 2

Portátil Lenovo Legion 5

16 GB de RAM | RTX 3060

\$850,000

Agregar al Carrito

PC Gamer Intel i7

32 GB de RAM | RX 6700 XT

\$1,200,000

Agregar al Carrito

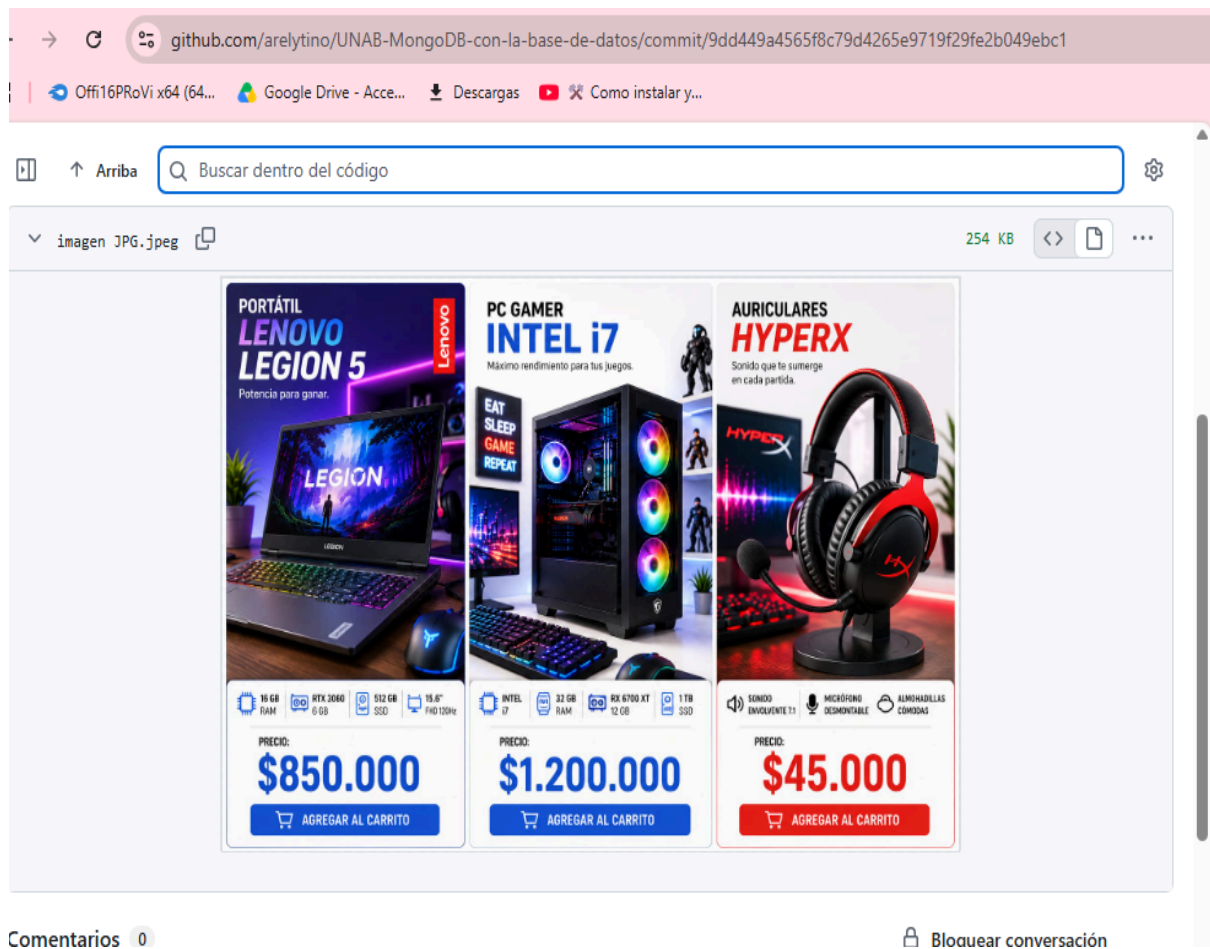
Auriculares HyperX

Sonido envolvente 7.1 | Micrófono

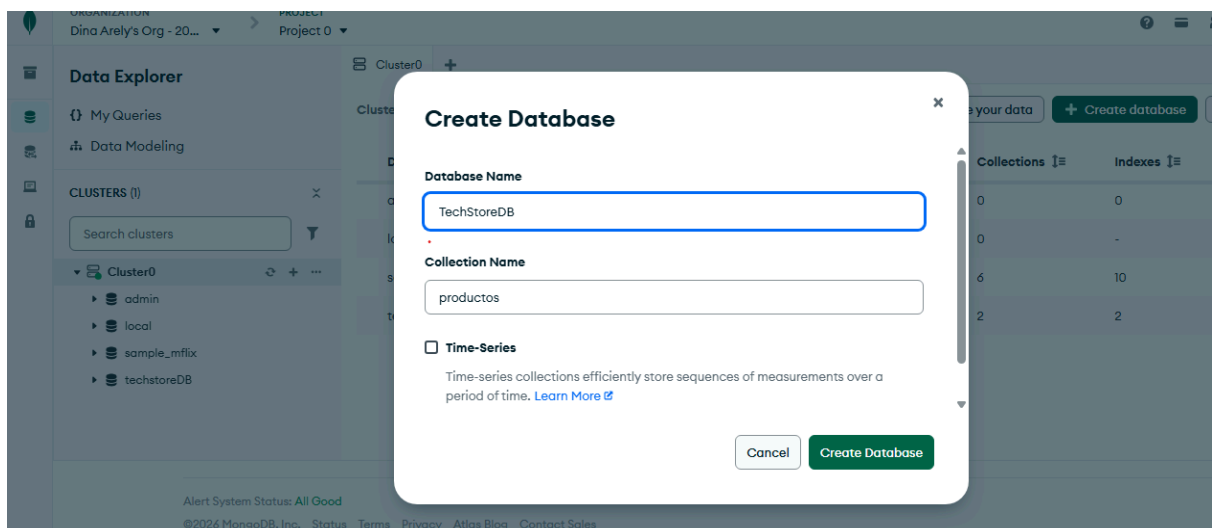
\$45,000

Agregar al Carrito

ANEXO #8 DETALLES DEL PRODUCTO



ANEXO # 9 CREANDO LA BASE DE DATOS EN MONGODB ATLAS.



ANEXO #10 CAPTURA CREACIÓN DEL CLUSTER

Connect to Cluster0



Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver	Version
Node.js ▼	6.7 or later ▼

2. Install your driver

Run the following on the command line

```
npm install mongodb
```



[View MongoDB Node.js Driver installation instructions.](#)

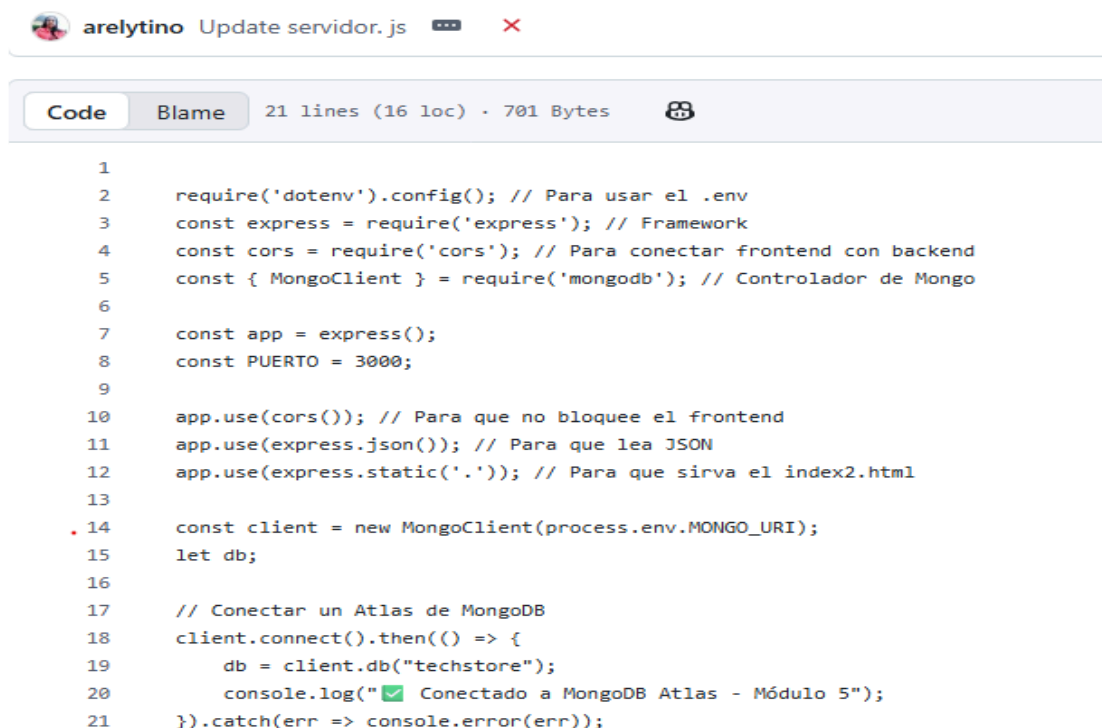
3. Add your connection string into your application code

Use this connection string in your application

ANEXO #10 INTEGRACIÓN CON EL BACKEND

El backend se desarrolló con Node.js + Express + Mongoose para conectar con MongoDB Atlas.

Código de conexión: [server.js](#)



```
1
2   require('dotenv').config(); // Para usar el .env
3   const express = require('express'); // Framework
4   const cors = require('cors'); // Para conectar frontend con backend
5   const { MongoClient } = require('mongodb'); // Controlador de Mongo
6
7   const app = express();
8   const PUERTO = 3000;
9
10  app.use(cors()); // Para que no bloquee el frontend
11  app.use(express.json()); // Para que lea JSON
12  app.use(express.static('.')); // Para que sirva el index2.html
13
14  const client = new MongoClient(process.env.MONGO_URI);
15  let db;
16
17  // Conectar un Atlas de MongoDB
18  client.connect().then(() => {
19    db = client.db("techstore");
20    console.log("✅ Conectado a MongoDB Atlas - Módulo 5");
21  }).catch(err => console.error(err));
```

ESPECIFICACIÓN DEL CÓDIGO ARCHIVO

```
const express = require('express');
```

```
const mongoose = require('mongoose');
```

```
const app = express();
```

```
app.use(express.json());
```

```
mongoose.connect('mongodb+srv://usuario:password@cluster.mongodb.net/techstore')
```

```
.then(() => console.log('Conectado a MongoDB'))
```

```
.catch(err => console.error(err));
```

```
app.listen(3000, () => console.log('Servidor en puerto 3000'));
```

Endpoints API REST:

Método	Endpoint	Descripción
GET	/api/productos	Listar todos los productos
POST	/api/carrito/total	Calcular el total del carrito
POST	/api/usuarios/login	Autenticar usuario
POST	/api/ventas	Registrar una nueva venta.

CÓDIGO ARCHIVO

```
"name": "techstore-2026-m5",  
"version": "1.0.0",  
"description": "Proyecto final Modulo 5 - TECHSTORE con Node.js y MongoDB",  
"main": "servidor.js",  
"scripts": {  
  "start": "node servidor.js",  
  "dev": "nodemon servidor.js",  
  "test": "jest"  
},  
"keywords": [  
  "techstore",  
  "ecommerce",  
  "nodejs",  
  "mongodb",  
  "unab"  
],
```

```

"author": "Arely Tino",

"license": "MIT",

"dependencies": {

  "express": "^4.18.2",

  "mongoose": "^8.0.0",

  "cors": "^2.8.5",

  "dotenv": "^16.3.1"

},

"devDependencies": {

  "nodemon": "^3.0.1",

  "jest": "^29.7.0"

}

}

```

NEXO #11 : PANEL DE DESPLIEGUE EN RENDER INCORPORANDO TECSTORE

